MTL-FLoRA: Federated Low-Rank Adaptation for Multi-Task Learning

Christian Jesinghaus[1] (✉), Joshua Heitbreder[1] (✉), and Julia Köpp[1] (✉)

Leibniz University of Hanover, Hanover Lower Saxony 30167, Germany
jesinghaus.christian@gmail.com
jheitbreder@gmx.de
julia.koepp@stud.uni-hannover.de

Abstract. We present **MTL-FLoRA**, a federated extension of **MTL-LoRA** for *multi-task* parameter-efficient fine-tuning of large transformer models that supports heterogeneous LoRA ranks across clients. Building on FLoRA's observation that naive FedAvg over LoRA factors introduces *aggregation noise*, we lift the MTL-LoRA adapter parameterization (A, Λ_t, B^i, w_t) into a federated setting and develop two server-side aggregation strategies: **(I)** an **exact client-wise stacking** construction with **matrix weighting** of the shared-knowledge, and **(II)** an **exact client-wise stacking** construction with **scalar mixture weights** and blockwise normalization. Both strategies stack client parameters into disjoint rank blocks (block-diagonal Λ_t and aligned A/B slices), thereby avoiding cross terms and ensuring that the global update matches the intended, data-proportional FedAvg update.

We evaluate federated multi-task tuning on GLUE tasks (MRPC and SST2) under IID and non-IID splits, comparing against centralized training and **FedIT** on two backbones: **RoBERTa** (encoder-only) and **TinyLlama** (decoder-only). Results reveal a strong dependence on heterogeneity and architecture: **Approach II** is most robust on TinyLlama (especially under non-IID splits) and in some configurations slightly surpasses the centralized baseline, while on RoBERTa **FedIT** remains a particularly strong reference method under non-IID data. To address the rank growth inherent to approach II, we study a linear-freeze ablation that retains competitive performance while improving scalability through reduced resource growth. In addition, we include an **average-w** ablation for the first approach that replaces stacked shared-knowledge weights with simple averaging, isolating the impact of weight stacking in the mixture mechanism.

Code - https://github.com/ChristianJesinghaus/MTL-FLoRA

Keywords: Federated Learning · Multi-Task Fine-Tuning · Low-Rank Adaptation

1 Introduction

Large pre-trained Transformer models are a standard backbone for many NLP tasks, but full fine-tuning is costly in memory, compute, and communication. Parameter-efficient fine-tuning (PEFT) methods such as Low-Rank Adaptation (LoRA) reduce this cost by training only small low-rank modules while keeping the backbone frozen [4]. Multi-task learning (MTL) can improve data efficiency and transfer across tasks. However, naive joint fine-tuning can cause *negative transfer*, especially when tasks compete for capacity in the same low-dimensional adapter space.

MTL-LoRA addresses this by separating *task-specific* and *shared* information within the low-rank parameterization [2]. It uses task-dependent transformations in rank space and mixes multiple shared up-projection matrices via task-conditioned weights, improving centralized multi-task fine-tuning at similar parameter budgets. Many instruction and task datasets are privacy-sensitive and distributed across devices or organizations, which motivates *federated learning* (FL) to enable collaboration without centralizing data [6]. Federated PEFT is attractive because clients only train and upload lightweight adapter parameters, reducing compute and communication.

A central challenge is aggregation. Averaging LoRA factors across clients can be inconsistent with averaging the implied weight updates, which introduces *aggregation noise* through cross terms and can harm convergence [3]. This becomes harder in federated multi-task settings with non-IID data, task heterogeneity, and potentially *heterogeneous client ranks*. FedIT shows federated instruction tuning with LoRA, but relies on FedAvg-style adapter averaging and inherits these limitations [1].

We introduce **MTL-FLoRA**, a federated extension of MTL-LoRA that combines task-specific vs. shared adaptation with stacking-based aggregation to avoid cross terms and support heterogeneous ranks [2, 3]. Each client trains an MTL-LoRA adapter locally. The server aggregates adapters by aligning rank blocks across A, Λ_t, and the shared B^i components and redistributes the result. We study two aggregation designs. One stacks all tensors directly, including shared-knowledge weighting parameters. The other uses an *exact* client-wise stacking with blockwise normalization that reproduces a FedAvg-style weighted average of local MTL-LoRA updates without cross terms.

We evaluate MTL-FLoRA on GLUE tasks (MRPC and SST2) under IID and non-IID splits and compare against centralized training and FedIT-style averaging [1]. We test both RoBERTa (encoder-only) and TinyLlama (decoder-only) backbones. We also analyze sensitivity to client weighting (p), adapter rank, the number of shared B matrices, and the trade-off between exact aggregation and rank growth across rounds.

1.1 Main Contributions

- **Federated multi-task PEFT with MTL-LoRA:** We introduce **MTL-FLoRA**, a federated formulation of MTL-LoRA for multi-task fine-tuning with privacy-preserving local training and lightweight adapter communication.
- **Two stacking-based aggregation strategies:** We propose and study two server-side aggregation designs for the full MTL-LoRA parameter set (A, Λ_t, B^i, w_t), extending stacking ideas from FLoRA to the multi-task adapter setting and enabling *heterogeneous client ranks*.
- **Approach I: Stacking LoRA parameters with shared knowledge weighting matrices** We extend FLoRA-style stacking to the full MTL-LoRA parameter set by aggregating A, Λ_t, and the shared B^i via aligned rank-block stacking, and by additionally stacking *matrix-valued* weighting parameters $w_t^i \in \mathbb{R}^{d\times r}$ to enable finer-grained, task-dependent mixing of shared knowledge in the federated setting. The approach does not introduce aggregation noise from cross terms and corresponds to the expected FedAvg weighted average of local MTL-LoRA updates.
- **Approach II: Stacking LoRA parameters with scalar shared knowledge weighting** We present a client-wise stacking construction with block-wise softmax normalization that is *exact* w.r.t. a FedAvg weighted average of local MTL-LoRA updates, thereby eliminating aggregation noise from cross terms by design.
- **Scalability ablation via linear-freeze stacking:** To address rank growth across rounds in exact stacking, we evaluate a *FreezeR* variant that freezes previously aggregated rank blocks and appends only new capacity per round, providing a controlled cost–performance trade-off.
- **Cross-architecture empirical study under heterogeneity:** We benchmark the proposed methods against centralized training and FedIT-style averaging on RoBERTa and TinyLlama under IID and non-IID splits, and we analyze sensitivity to rank and client-weight scaling, highlighting when stacking helps and when simpler averaging remains competitive.

2 Related Work

2.1 Low-Rank Adaptation for Multi-Task Learning

MTL-LoRA LoRA-style adapters are highly parameter-efficient, yet joint multi-task fine-tuning can suffer from negative transfer when task updates are compressed into the same low-dimensional intrinsic space. MTL-LoRA addresses this issue by explicitly separating task-specific and shared information in the adapter design. Concretely, it introduces a *task-specific* learnable transformation $\Lambda_t \in \mathbb{R}^{r\times r}$ in the low-rank space, and models *shared* knowledge via multiple up-projection matrices $\{B_i\}_{i=1}^n$ that are combined through a task-dependent weighting vector w_t (softmax with temperature) [2]. Across diverse benchmarks (including NLU, GLUE, commonsense reasoning, image–text understanding, and

an industrial ads-relevance dataset), MTL-LoRA reports consistent improvements over LoRA variants at comparable parameter budgets [2]. In our setting, MTL-LoRA provides the centralized multi-task adapter structure that we lift into a federated environment.

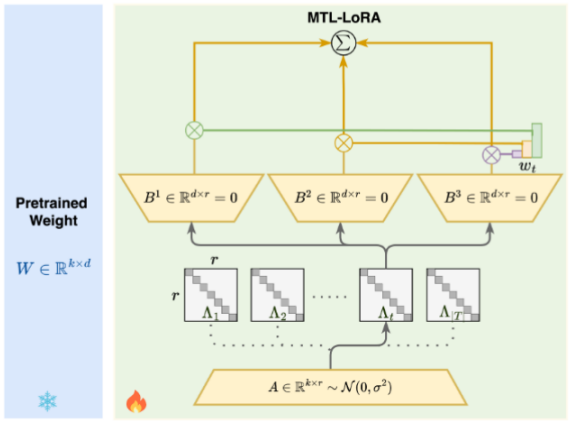

Fig. 1. Overall architecture of MTL-LoRA: a shared down-projection A, a task-specific transformation Λ_t in the low-rank space, and multiple up-projection matrices $\{B_i\}$ mixed via task-dependent weights w_t. [2].)

2.2 Federated Fine-Tuning with Low-Rank Adapters

FLoRA A common baseline for federated LoRA fine-tuning aggregates client adapters via FedAvg-style averaging of the low-rank factors. FLoRA shows that independently averaging A_k and B_k is mathematically inconsistent with aggregating the actual updates $B_k A_k$, which introduces *aggregation noise* through cross terms and can harm convergence [3]. To eliminate this effect, FLoRA proposes a stacking-based construction of global LoRA modules that is theoretically accurate and naturally supports *heterogeneous* client ranks (since stacking does not require identical ranks across clients) [3]. This stacking principle directly motivates our server-side aggregation.

FedIT FedIT is an early framework for *federated instruction tuning* of large language models, motivated by the fact that instruction data are often privacy-sensitive and naturally distributed across many data owners [1]. To make federated tuning feasible on resource-constrained clients, FedIT adopts parameter-efficient tuning via LoRA: each round, the server selects a subset of clients, clients fine-tune only the LoRA adapter parameters on local instructions while keeping

the backbone frozen, and the server aggregates the uploaded adapter updates in a FedAvg-style manner [1]. The work emphasizes the role of instructional heterogeneity in FL and reports quality gains under GPT-4 based auto-evaluation. It also releases an experimentation framework (Shepherd) to support future federated LLM tuning research [1].

3 Approach

The MTL-FLoRA architecture (Fig. 2) introduces a federated setup of the MTL-LoRA [2] architecture. MTL-LoRA offers a possibility to separate the task-specific knowledge into multiple low-dimensional spaces. This centralized approach improves upon diverse benchmarks in case of multi-task finetuning. The model finetunes their LoRA matrices A, Λ_t, B^i, w_t^i. Besides the A matrix, the model trains T different task specific Λ_t matrices for saving the task-specific knowledge, multiple B^i matrices for saving the shared knowledge between the tasks and a weight w_t^i for weighting the shared knowledge in B^i differently per task. In inference the MTL-LoRA model computes the output of task t as $h_t = Wx_t + \sum_{i=1}^{n} \frac{\exp(w_t^i/\tau)B_i}{\sum_{j=1}^{n}\exp(w_t^j/\tau)} \Lambda_t A x_t$ [2] using the hyperparameter τ to control the sharpness of the weight distribution.

Our MTL-FLoRA architecture extends the central MTL-LoRA architecture for the federated multi-task finetuning. Each client is setup with the MTL-LoRA architecture and finetunes on the same T tasks, with different splits on non-IID task datasets. The clients can have heterogeneous rank sizes. After finetuning on the local datasets each clients sends their weights A, Λ_t, B^i, w_t^i to the server. The server aggregates the weights and then redistributes them back to the clients. We tested two stacking based aggregation strategies, one using matrices for weighting the shared knowledge and one using scalar values. The following sections explain why both strategies do not introduce bias through cross terms and exactly correspond to the expected global model update.

3.1 Aggregation Stategy I: Stacking LoRA parameters with shared knowledge weighting matrices

Our aggregation approach builds upon the FLoRA [3] approach by extending it for handling multiple LoRA parameters. FLoRA states that the FedIT framework [1], introduces aggregation noise though cross-terms by simply averaging the LoRA A and B matrices. Their approach eliminates this error though horizontally stacking the clients B matrices and vertically stacking the A (along the rank dimension). This additionally enables heterogenes rank sizes across clients. Due to stacking the parameters the rank size is increases by the sum of the client ranks of the last round in each communication round. In adaptation to the MTL-LoRA architecture, which includes multiple LoRA parameter A, Λ_t, B^i, w_t^i, instead of just A and B, we also stacked the task specific Lambda matrices and

the shared knowledge weighting parameters. We also changed the scalar weighting to a weighting matrices of size $d \times r$ to allow finer client adjustments of the shared knowledge B^i matrices. The server stacks the client A matrices vertically (by rank dimension) and multiplies a scaling factor $p_k = \frac{\text{len}(D_k)}{\text{len}\left(\sum_{k=0}^{K} D_k\right)}$ determined by the relative size the local dataset of each client compared to the global size [3]. The B^i are stacked horizontally (by rank dimension). The Λ matrices are diagonally stacked (by rank dimension) to match the corresponding parts in the A and B matrices and zero-padded to avoid bias through cross-terms. The weights w_t^i are stacked horizontally (by rank dimension) to match the alignment of the B^i for element-wise multiplication.

We use a simple example to motivate our changes in stacking:

Notation: $\cdot$ scalar multiplication, $\odot$ elementwise multiplication, $*$ matrix multiplication, exp() Hadamard exponential, / (elementwise division)

In each communication round each client trains their weights

$$(Client1)[A, \Lambda_t, B^i, w_t^i], (Client2)[A, \Lambda_t, B^i, w_t^i] \tag{1}$$

The local model updates are computed as the corresponding product:

$$\Delta W_{t(Client)} = \sum_{i=1}^{n} \frac{\exp\left(w_t^i/\tau\right) \odot B^i}{\sum_{j=1}^{n} \exp\left(w_t^j/\tau\right)} \quad * \Lambda_t * A \tag{2}$$

Standard FedAvg [6] receives the global model update ΔW_t though weighted average, therefore the expected global model update ΔW_t is the weighted average from the clients $\Delta W_{t(Client)}$:

$$\Delta W_t = p_{(Client1)} \cdot \Delta W_{t(Client1)} + p_{(Client2)} \cdot \Delta W_{t(Client2)} \tag{3}$$

The following argumentation shows that our stacking based method produces the same global model update as FedAvg.
After local finetuning the server aggregates and redistributed the client weights:

$$\begin{aligned} A &= \begin{bmatrix} p_{(C1)} \cdot A_{(C1)} \\ p_{(C2)} \cdot A_{(C2)} \end{bmatrix}, B^i = \begin{bmatrix} B^i_{(C1)} & B^i_{(C2)} \end{bmatrix} \\ \Lambda_t &= \begin{bmatrix} \Lambda_{t(C1)} & 0...0 \\ 0..0 & \Lambda_{t(C2)} \end{bmatrix}, w_t^i = \begin{bmatrix} w^i_{t(C1)} w^i_{t(C2)} \end{bmatrix} \end{aligned} \tag{4}$$

The dimensions of the stacked parameters are:
$A \in \mathbb{R}^{(r_{(C1)}+r_{(C2)}) \times k}$ - The A are stacked in the rank dimension.
$B^i \in \mathbb{R}^{d \times (r_{(C1)}+r_{(C2)})}$ - Each B^i matrix is stacked in the rank dimension.
$\Lambda_t \in \mathbb{R}^{(r_{(C1)}+r_{(C2)}) \times (r_{(C1)}+r_{(C2)})}$ - The matrices are stacked diagonally.
$w_t^i \in \mathbb{R}^{d \times (r_{(C1)}+r_{(C2)})}$ - The weights of each clients are the size $d \times r$ and then stacked in the rank dimension to match the size of the B^i matrices.

The global model update ΔW_t for the stacked weights therefore is:

$$\begin{aligned}\Delta W_t = &\sum_{i=1}^{n} \frac{\exp\Big(\Big[w^i_{t(C1)}\ w^i_{t(C2)}\Big]/\tau\Big) \odot \Big[B^i_{(C1)}\ B^i_{(C2)}\Big]}{\sum_{j=1}^{n}\exp\Big(\Big[w^j_{t(C1)}\ w^j_{t(C2)}\Big]/\tau\Big)} \\ &* \begin{bmatrix} \Lambda_{t(C1)} & 0...0 \\ 0..0 & \Lambda_{t(C2)} \end{bmatrix} * \begin{bmatrix} p_{(C1)} \cdot A_{(C1)} \\ p_{(C2)} \cdot A_{(C2)} \end{bmatrix}\end{aligned} \tag{5}$$

We define:

$$\begin{aligned} B_{stacked} &:= \sum_{i=1}^{n} \frac{\exp\Big(\Big[w^i_{t(C1)}\ w^i_{t(C2)}\Big]/\tau\Big) \odot \Big[B^i_{(C1)}\ B^i_{(C2)}\Big]}{\sum_{j=1}^{n}\exp\Big(\Big[w^j_{t(C1)}\ w^j_{t(C2)}\Big]/\tau\Big)} \\ \Lambda_{stacked} &:= \begin{bmatrix} \Lambda_{t(C1)} & 0...0 \\ 0..0 & \Lambda_{t(C2)} \end{bmatrix} \\ A_{stacked} &:= \begin{bmatrix} p_{(C1)} \cdot A_{(C1)} \\ p_{(C2)} \cdot A_{(C2)} \end{bmatrix} \end{aligned} \tag{6}$$

$$\Delta W_t = B_{stacked} * \Lambda_{stacked} * A_{stacked} \tag{7}$$

$B_{stacked}$ can be simpified as follows:

$$B_{stacked} := \sum_{i=1}^{n} \frac{\Big[\exp\Big(w^i_{t(C1)}/\tau\Big)\ \exp\Big(w^i_{t(C2)}/\tau\Big)\Big] \odot \Big[B^i_{(C1)}\ B^i_{(C2)}\Big]}{\Big[\sum_{j=1}^{n}\exp\Big(w^j_{t(C1)}/\tau\Big)\ \sum_{j=1}^{n}\exp\Big(w^j_{t(C2)}/\tau\Big)\Big]} \tag{8}$$

$$B_{stacked} := \sum_{i=1}^{n} \left[\frac{exp\Big(w^i_{t(C1)}/\tau\Big)}{\sum_{j=1}^{n}\exp\Big(w^j_{t(C1)}/\tau\Big)}\ \frac{\exp\Big(w^i_{t(C2)}/\tau\Big)}{\sum_{j=1}^{n}\exp\Big(w^j_{t(C2)}/\tau\Big)}\right] \odot \Big[B^i_{(C1)}\ B^i_{(C2)}\Big] \tag{9}$$

$$B_{stacked} := \left[\sum_{i=1}^{n}\frac{exp\Big(w^i_{t(C1)}/\tau\Big)}{\sum_{j=1}^{n}\exp\Big(w^j_{t(C1)}/\tau\Big)} \odot B^i_{(C1)}\ \sum_{i=1}^{n}\frac{\exp\Big(w^i_{t(C2)}/\tau\Big)}{\sum_{j=1}^{n}\exp\Big(w^j_{t(C2)}/\tau\Big)} \odot B^i_{(C2)}\right] \tag{10}$$

As mentioned above:

$$\Delta W_t = B_{stacked} * \Lambda_{stacked} * A_{stacked} \tag{11}$$

Therefore:

$$\begin{aligned}\Delta W_t = &\left[\sum_{i=1}^{n}\frac{exp\Big(w^i_{t(C1)}/\tau\Big)}{\sum_{j=1}^{n}\exp\Big(w^j_{t(C1)}/\tau\Big)} \odot B^i_{(C1)}\ \sum_{i=1}^{n}\frac{\exp\Big(w^i_{t(C2)}/\tau\Big)}{\sum_{j=1}^{n}\exp\Big(w^j_{t(C2)}/\tau\Big)} \odot B^i_{(C2)}\right] \\ &* \begin{bmatrix} \Lambda_{t(C1)} & 0...0 \\ 0..0 & \Lambda_{t(C2)} \end{bmatrix} * \begin{bmatrix} p_{(C1)} \cdot A_{(C1)} \\ p_{(C2)} \cdot A_{(C2)} \end{bmatrix}\end{aligned} \tag{12}$$

If we multiply the matrices we get:

$$\begin{aligned} \Delta W_t = p_{(C1)} \cdot \sum_{i=1}^{n} \frac{exp\Big(w^i_{t(C1)}/\tau\Big)}{\sum_{j=1}^{n} \exp\Big(w^j_{t(C1)}/\tau\Big)} \odot B^i_{(C1)} * \Lambda_{t(C1)} * A_{(C1)} + \\ p_{(C2)} \cdot \sum_{i=1}^{n} \frac{\exp\Big(w^i_{t(C2)}/\tau\Big)}{\sum_{j=1}^{n} \exp\Big(w^j_{t(C2)}/\tau\Big)} \odot B^i_{(C2)} * \Lambda_{t(C2)} * A_{(C2)} \end{aligned} \tag{13}$$

This exactly corresponds to:

$$\Delta W_t = p_{(Client1)} \cdot \Delta W_{t(Client1)} + p_{(Client2)} \cdot \Delta W_{t(Client2)} \tag{14}$$

The argumentation proofs that our stacking approach is equal to the weighted average of each clients local updates $\Delta W_{t(Client)}$ and does not introduces bias through cross terms.

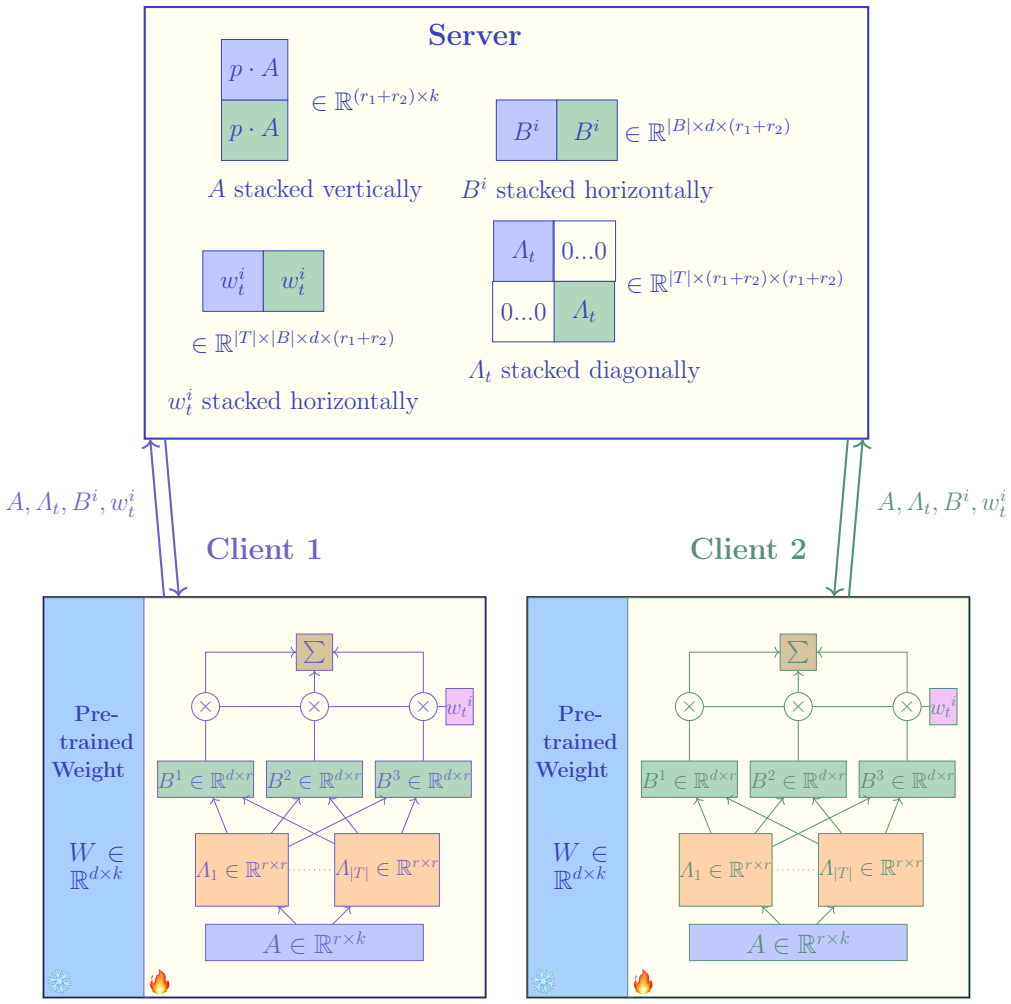

Fig. 2. Aggregation Stategy I
The MTL-FLoRA architecture introduces a federated setup for the centralized MTL-LoRA architecture [2]. Each client is setup with the MTL-LoRA architecture and finetunes on the same T tasks, each client contains an A matrix, T task-specific Λ matrices, multiple B matrices for shared task knowledge, and a task specific weight matrix w_t^i of size $d \times r$ for weighting each of the B matrices. r is the LoRA-rank of the client. After finetuning on the tasks locally, the clients sending their weights to the server. The server stackes the A matrices from the clients vertically (by rank dimension) and scales them by client size. The B^i are horizontally (by rank dimension) stacked. The Λ_t are stacked diagonally and the w_t^i are stacked horizontally (by rank dimension) to match the size of the B matrices. The server redistributed the weights back to the clients. Our stacking approach extends the FLoRA [3] approach by adjusting it to the MTL-LoRA architecture, which contains multiple parameter A, Λ_t, B^i, w_t^i for handling multi-task finetuning.

3.2 Aggregation Strategy II: Stacking LoRA parameters with scalar shared knowledge weighting

We also consider an *exact* stacking construction that reproduces a FedAvg-style weighted average of the *local MTL-LoRA updates* without introducing cross terms (cf. the aggregation-noise observation in FLoRA [3]). This variant uses scalar weighting of the shared knowledge matrices.

Local update. Let $c \in \{1, \dots, K\}$ index clients and $t \in \{1, \dots, T\}$ tasks. Client c uses rank r_c and trains $A_c \in \mathbb{R}^{r_c \times k}, \Lambda_{c,t} \in \mathbb{R}^{r_c \times r_c}, \{B_{c,i}\}_{i=1}^{n}$, $B_{c,i} \in \mathbb{R}^{d \times r_c}$ with task-dependent mixture weights from $w_{c,t} \in \mathbb{R}^n$ via a temperature-softmax ($\tau > 0$):

$$\alpha_{c,t,i} = \frac{\exp(w_{c,t,i}/\tau)}{\sum_{j=1}^{n} \exp(w_{c,t,j}/\tau)}. \tag{15}$$

The resulting local adapter update is

$$\Delta W_{c,t} = \sum_{i=1}^{n} \alpha_{c,t,i}\, B_{c,i}\, \Lambda_{c,t}\, A_c \in \mathbb{R}^{d \times k}. \tag{16}$$

FedAvg target on updates. With local sample counts $|D_c|$ and FedAvg weights

$$p_c = \frac{|D_c|}{\sum_{u=1}^{K} |D_u|}, \tag{17}$$

the desired aggregated update is

$$\Delta W_t^{\text{FedAvg}} = \sum_{c=1}^{K} p_c\, \Delta W_{c,t}. \tag{18}$$

Exact stacking (heterogeneous ranks, no cross terms). Define the total rank $r_{\text{tot}} = \sum_{c=1}^{K} r_c$ and rank offsets $s_c = \sum_{u<c} r_u$ (so $s_1 = 0$). We stack clients into disjoint rank blocks:

(i) Stacked down-projection.

$$A^{\text{stack}} = \begin{bmatrix} p_1 A_1 \\ p_2 A_2 \\ \vdots \\ p_K A_K \end{bmatrix} \in \mathbb{R}^{r_{\text{tot}} \times k}. \tag{19}$$

(ii) Block-diagonal task transforms.

$$\Lambda_t^{\text{stack}} = \text{blkdiag}\big(\Lambda_{1,t}, \Lambda_{2,t}, \dots, \Lambda_{K,t}\big) \in \mathbb{R}^{r_{\text{tot}} \times r_{\text{tot}}}. \tag{20}$$

(iii) Client-wise stacked up-projections. For each (c, i), embed $B_{c,i}$ into the c-th block (zeros elsewhere):

$$B_{c,i}^{\text{stack}} = \begin{bmatrix} 0_{d \times s_c} & B_{c,i} & 0_{d \times (r_{\text{tot}} - s_c - r_c)} \end{bmatrix} \in \mathbb{R}^{d \times r_{\text{tot}}}. \tag{21}$$

Compared to the usual n global matrices, this yields $K \cdot n$ matrices $B_{c,i}^{\text{stack}}$, which prevents interaction between different clients' rank blocks.

Blockwise softmax. To preserve each client's local mixture exactly, we do not normalize across all $K \cdot n$ blocks. Instead, we apply a softmax within each client block:

$$\alpha_{c,t,i}^{\text{stack}} = \frac{\exp(w_{c,t,i}/\tau)}{\sum_{j=1}^{n} \exp(w_{c,t,j}/\tau)} \;=\; \alpha_{c,t,i}. \tag{22}$$

Global stacked update and exactness. Define the stacked update as

$$\Delta W_t^{\text{stack}} = \sum_{c=1}^{K} \sum_{i=1}^{n} \alpha_{c,t,i}^{\text{stack}}\; B_{c,i}^{\text{stack}}\; \Lambda_t^{\text{stack}}\; A^{\text{stack}}. \tag{23}$$

Since rank blocks are disjoint, $B_{c,i}^{\text{stack}} \Lambda_t^{\text{stack}} A^{\text{stack}} = B_{c,i} \Lambda_{c,t} (p_c A_c)$, and therefore

$$\Delta W_t^{\text{stack}} = \sum_{c=1}^{K} p_c \Big(\sum_{i=1}^{n} \alpha_{c,t,i}\; B_{c,i} \Lambda_{c,t} A_c \Big) = \sum_{c=1}^{K} p_c\, \Delta W_{c,t} = \Delta W_t^{\text{FedAvg}}. \tag{24}$$

Hence, client-wise stacking is exact w.r.t. the FedAvg target (18) and introduces no cross terms.

Ablation: Linear-freeze stacking. To mitigate the rapid parameter growth of exact client-wise stacking across multiple FL rounds, we evaluate a *linear-freeze* variant. After the first stacking round, the current global adapter is treated as a frozen prefix: in each subsequent round, every client expands the adapter by appending a small *new* rank block (and corresponding new B matrices) while keeping all previously aggregated blocks fixed. Concretely, clients optimize only the newly appended parameters, and the server aggregates by inserting each client's new block into disjoint rank/B slices (preserving the no-cross-term property of blockwise stacking) while leaving the frozen prefix unchanged. As a result, the effective adapter size grows *linearly* with the number of rounds rather than exponentially, yielding a controlled-capacity ablation of Aggregation Strategy II.

4 Experiments

We conduct a series of experiments to explore the effectiveness of MTL-LoRA in a federated setting. Finally, we analyze the performance of MTL-FLoRA under different hyperparameter configurations. The limitations section of the Flora paper notes that their evaluation is restricted to LLaMA models. In contrast, we conduct experiments on both RoBERTa and TinyLLaMA to compare encoder-only and decoder-only architectures, providing a broader perspective on model performance across different transformer types.

4.1 Setup: RoBERTa - Approach I

To validate the performance of MTL-LoRA in a federated multi-task setting, we fine-tune three models on exemplary tasks of the GLUE dataset, which served

as a benchmark in the original MTL-LoRA paper [2].
We use RoBERTa as a backbone model for the following experiments. The baseline model was trained in a centralized setting. The other two models were trained in a federated setting, where each client fine-tuned on both tasks. With our federated models we compare two aggregation strategies, FLoRA and FedIT, to assess whether our FLoRA based aggregation approach improves performance over naive averaging. As an ablation study we conducted the experiments on full Flora stacking and Flora stacking with averaging the weights w_i^t.
Our evaluation was conducted on the MRPC (Microsoft Research Paraphrase Corpus) and SST2 (Stanford sentiment treebank) datasets. The centralized model was trained for two epochs on the whole training splits for both tasks. For the federated variants, each client was trained on half of the data for one epoch per round, with two communication rounds in total. To assess the impact of IID vs. non-IID client splits, the federated experiments were conducted with dirichlet distributions of 0.1 and 10.
The following hyperparameter configurations were explored for the RoBERTa experiments:

1. $[1\cdot\text{Epoch}, 2\cdot\text{Epochs}, 3\cdot\text{Epochs}]\times[1\cdot\text{FL-Round}, 2\cdot\text{FL-Rounds}, 3\cdot\text{FL-Rounds}]\times[2\cdot\text{B}, 3\cdot\text{B}]$
2. [Best config. from 1.] $\times$ [Four inital LoRA Ranks per Client, Eight inital LoRA Ranks per Client, 16 initial LoRA Ranks per Client]
3. [Best config. from 2.]$\times$[P Factor $= 0.001$, P Factor $= 0.01$, P Factor $= 0.05$, P Factor $= 0.025$, P Factor $= 0.5$, P Factor $= 1.0$, P Factor $= 1.25$, P Factor $= 1.5$]
4. [Best config. from 3.] $\times$ [$\alpha = 0.1, \alpha = 10$]

4.2 Setup: Tinyllama - Approach II

The second approach was implemented on the TinyLlama-1.1B-Chat-v1.0 Causal Language Model, which is based on LLaMa, contains 1.1 billion parameters and has an approximate size of 2.2GB. Training was conducted on either Nvidia A100 or V100 GPUs. The second approach's performance is displayed against, a centralized training, FedIT, the averaging-w approach and against the second approach with additional freezing of the previous ranks. As the possible search space for optimal hyperparameter configurations too large for a non-automated Setting, we had to limit it to specifig configurations. These configurations were the following:

1. $[1\cdot\text{Epoch}, 2\cdot\text{Epochs}, 3\cdot\text{Epochs}]\times[1\cdot\text{FL-Round}, 2\cdot\text{FL-Rounds}, 3\cdot\text{FL-Rounds}]\times[2\cdot\text{B}, 3\cdot\text{B}]$
2. [Best config. from 1.] $\times$ [Four inital LoRA Ranks per Client, Eight inital LoRA Ranks per Client]
3. [Best config. from 2.]$\times$[P Factor $= 0.001$, P Factor $= 0.01$, P Factor $= 0.05$, P Factor $= 0.5$, P Factor $= 1.0$, P Factor $= 1.25$, P Factor $= 1.5$]
4. [Best config. from 3.] $\times$ [$\alpha = 0.1, \alpha = 10$]

With α meaning the Dirichlet α value, for the data Distribution between clients, P Factor meaning the Aggregation scaling factor P per Client, B standing for the B Matrices, FL-Round for Federated Learning Rounds.

The above shown configuration space was used for training and validation of each approach, if applicable. Overall the best results of the proposed approach II for each step of this Search Space were achieved under the following combination:

1. Three Epochs, Three FL-Rounds(if applicable), three B-Matrices
2. Eight initial Lora Ranks per Client
3. P Factor = 1.0
4. $\alpha = 10$

For FedIT, approach II with freezed R and averaged-w, the optimal parameter configuration mostly stayed the same. For example approach II with freezed Rs optimal Factor P value was 1.5, but everything else stayed untouched. A listing of the concrete configurations per approach can be found in the Appendix under Section 7. These Settings were used for each comparison, except it is mentioned otherwise. Furthermore it is worth mentioning, that only validation results are used, as this is also done in the original MTL-LoRA Paper [2]. Therefore we sticked with that to keep the results compareable.

4.3 Experimental Results

We evaluate federated multi-task tuning on the GLUE tasks MRPC and SST2, reporting validation-set results under IID and non-IID client splits. We compare against centralized training and **FedIT** on two backbones: **RoBERTa** (encoder-only) and **TinyLlama** (decoder-only). This aligns with the MTL-LoRA paper, which reports GLUE performance on the validation set only. For comparability, we follow the same protocol.

4.4 Results for Approach I and RoBERTa

Table 1 shows the GLUE validation results of centralized MTL-LoRA and our three stacking approaches in the federated setting with IID and non-IID dataset splits. The ablation study of our approach FLoRA-without-w uses averaging of the shared knowledge weights w_t^i. On average, FedIT achieves the highest accuracies in both IID and non-IID scenarios. On the MRPC task, the centralized, FLoRA-without-w and FedIT models achieve very similar scores for IID, but FedIT scores higher in non-IID. On the SST2 task, FLoRA-stacked-w outperforms the other models. FLoRA-without-w strongly degrades in non-IID, achieving an F1 score of zero on MRPC.
The FLoRA client weight p significantly influences the model performance. For FLoRA-without-w, a p value of around 1.25 is optimal. For full stacked Flora

(approach 1), the optimal p is slightly lower at aroung 1.0. Figure 3 also shows the performance of the federated models for different LoRA rank sizes r. For FLoRA-without-w, r=8 is optimal. full stacked Flora (approach 1) and FedIT score best at lower ranks with performance decreasing for r>4.

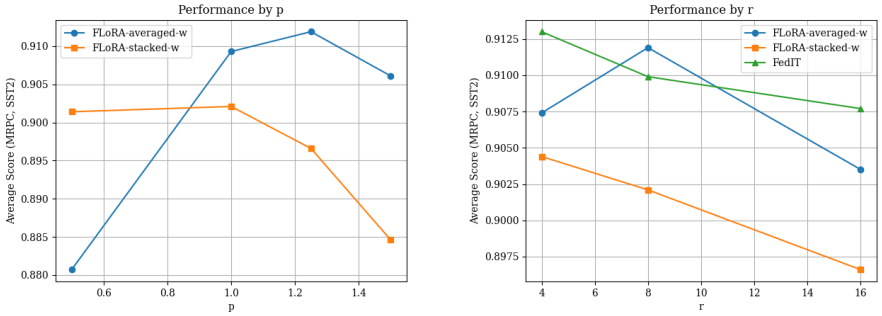

Fig. 3. Performance of RoBERTa-MTL-LoRA with different stacking approaches on the GLUE benchmark. It shows the performance of FloRA-stacked-w (approach 1), the ablation FloRA-averaged-w (without stacked w) and FedIT. Left: Performance by p. Right: Performance by r.

Method	MRPC (Acc.)	MRPC (F1)	SST2 (Acc.)	Avg.
Centralized	0.8799	0.9133	0.9358	0.9078
IID:				
FLoRA-approach-1	0.8578	0.8982	0.945	0.9014
- without stacked w	0.8799	0.9123	0.9438	0.9119
FedIT	**0.8799**	**0.913**	**0.9461**	**0.9130**
non-IID:				
FLoRA-approach-1	0.5294	0.6308	**0.8291**	0.6793
- without stacked w	0.3162	0.000	0.4908	0.4035
FedIT	**0.6863**	**0.8134**	0.7431	**0.7147**

Table 1. Results on MRPC and SST2 from the GLUE benchmark under IID and non-IID settings for the first approach and RoBERTa Model. FLoRA-approach-1 is our full stacked version. The ablation study without stacked w averages the weighting of the shared knowledge weights instead of stacking.

4.5 Results for Approach II and Tinyllama

Table 2 displays eachs approach IID($\alpha = 10$) and non-IID($\alpha = 0.1$) results for their runs under the best parameter configuration. The results show, that the

approach II based stacking performs better compared to all other variants for IID, as well as non-IID. Eventhough all variants perform competitive in the IID setting with only a few percentage, or even subpercentage points differences in average accuracy, the non-IID results reveal significant differences. For a non-IID setting FedIT performs 11.6 percent worse, than the second approach (without freezing R). Furthermore approach II Stacking is the only Implementation that results in a better average accuracy, than the centralized implementation. Another interesting investigation is, that approach II with additional freezing of R performs just minorly worse, than without freezing. This is especially true for the non-IID case. As the case of activated freezing of R has linear scaling of Lora Ranks, instead of exponential, this might be a good alternative for settings with limited Ressources.

Method	MRPC (Acc.)	MRPC (F1)	SST2 (Acc.)	Avg.
Centralized	83.6	88.1	96.0	89.8
IID:				
FLoRA-approach-2	**84.3**	**89.0**	**96.0**	**90.1**
+ freezeR	80.6	86.9	95.1	87.9
FLoRA-approach-1				
- without stacked w	82.4	87.8	95.6	89.0
FedIT	83.8	88.6	95.4	89.6
non-IID:				
FLoRA-approach-2	36.0	15.5	87.4	**61.7**
FLoRA-approach-1				
- without stacked w	35.3	13.7	**88.0**	61.6
FedIT	**47.1**	**45.2**	53.2	50.1

Table 2. Results on MRPC and SST2 from the GLUE benchmark under IID and non-IID settings for all approaches tested on Tinyllama Model. Each result was generated under the best parameter configuration from the above mentioned search space.

In Table 3 one can see, how the second approach with exponential rank growth compares to it's variant with linear rank growth under variations of the scaling factor P and IID data split. This is also displayed in Figure 4, where additionally the averaged-w approach is displayed. When P is set to a value smaller than 1.0 all models experience degradation. Especially after a value of 0.5 the average accuracy declines significantly. Although both approach II variants retain a much higher average accuracy than the average-w approach, indicating a higher stability of these variants under P variation.

Method	p	MRPC (Acc.)	MRPC (F1)	SST2 (Acc.)	Avg.
FLoRA-stacked	0.001	68.4	81.2	64.7	66.5
	0.01	68.4	81.1	63.9	66.1
	0.5	75.7	84.3	94.6	85.2
	1.0	**84.3**	**89.0**	**96.0**	**90.1**
	1.5	68.4	81.2	94.4	81.4
FLoRA-stacked + freezed R	0.001	68.6	80.1	72.7	70.7
	0.01	66.9	78.5	65.9	66.4
	0.5	77.5	83.0	95.3	86.4
	1.0	78.9	84.6	**95.4**	87.2
	1.5	**80.6**	**86.9**	95.1	**87.9**

Table 3. Effect of the aggregation scaling factor p on MRPC and SST2 for FLoRA-stacked approach II and FLoRA-stacked approach II + freezed R. Although the variant without freezing R performs better under optimal parameter Settings, with freezing R still performs competitivly good. For some P values it even outperforms the non-freezing approach.

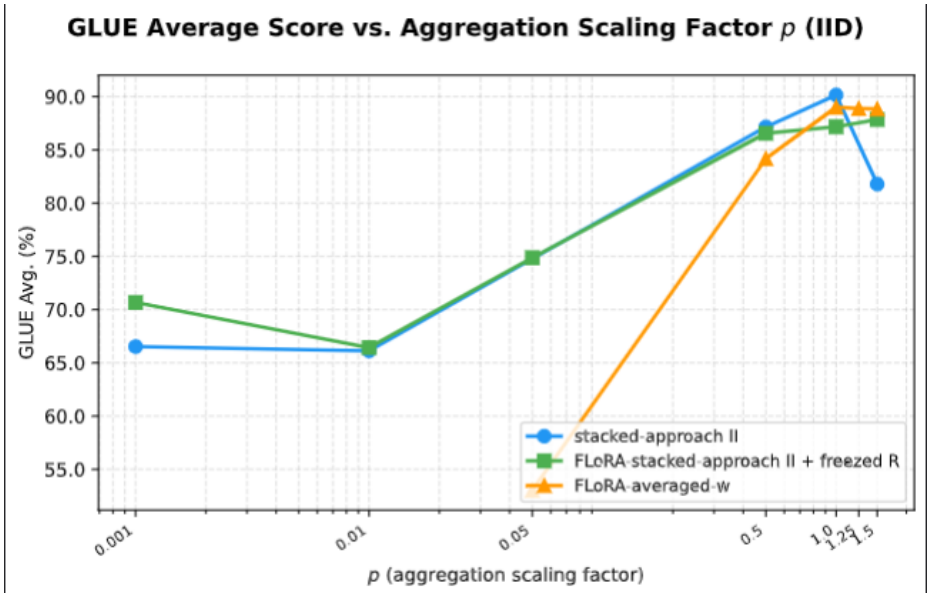

Fig. 4. The Figure shows the behavior of approach II with and without freezing and of the averaged-w approach under client p variations and IID data split. The plain version of approach II peaks at client p = 1.0, with around 90%, while additional freezing of R makes the approach peak at client p 1.5 and the averaged-w approach forms a plateau between 1.0 to 1.5.

After inspecting the overall behaviors under non-IID/IID settings and variations of P, we also conducted a look into how the approaches perform within the two different GLUE Tasks for IID and non-IID, in order to find out where the discrimination between the approaches is rooted. Figure 5 and Figure 6 show how the different approaches perform according to their MRPC and SST2 accuracy. In the IID setting one can observe, that all approaches perform similarly within the SST2 accuracy and almost all discrimination between the approaches is achieved, due to a difference in the MRPC accuracy. On the contrary Figure 6 shows, that in an non-IID setting, the significant advantage of approach II and averaged-w comes from a much higher accuracy in the SST2 Dimension! Here FedIT degrades significantly in comparison to the other two approaches, while it retains a slightly higher MRPC accuracy. This might be an indication, that the Outperformance of the stacking approach in the non IID Setting heavily depends on the structure of the dataset.

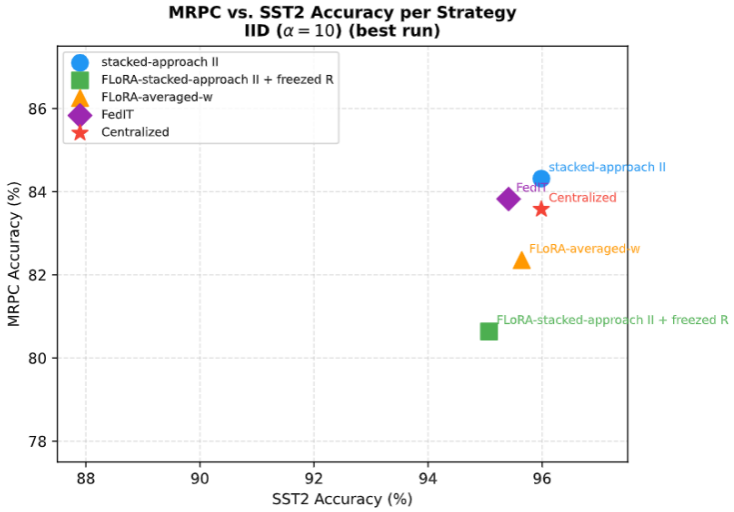

Fig. 5. Scatter plot of MRPC accuracy vs. SST2 accuracy for all federated strategies on Tinyllama under IID data distribution ($\alpha = 10$), showing the best-performing run per strategy. All approaches achieve competitive performance, with FLoRA-stacked reaching the highest MRPC accuracy while FLoRA-stacked approach II serves as an upper bound.

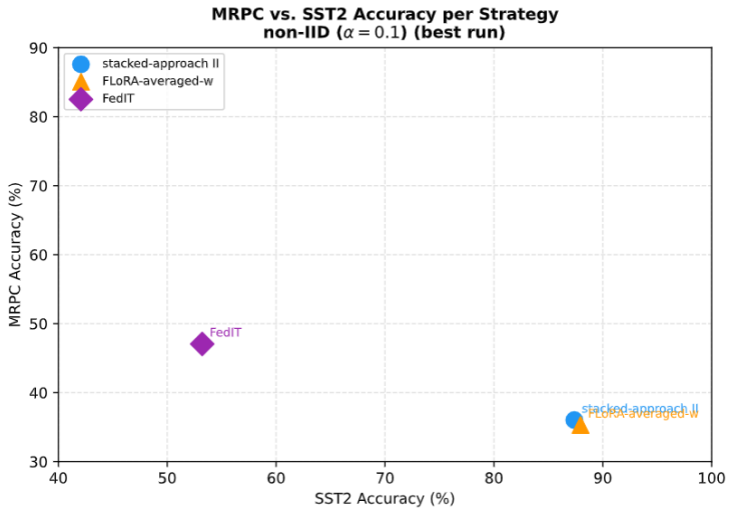

Fig. 6. Scatter plot of MRPC accuracy vs. SST2 accuracy for approach II, averaged-w and FedIT on Tinyllama under non-IID data distribution ($\alpha = 0.1$). Performance degrades significantly across all strategies compared to the IID setting, with FLoRA-stacked approach II and averaged-w showing the strongest robustness to data heterogeneity. Eventhough FedIT shows a slightly better performance in the MRPC dimension, it performs in the SST2 Dimension a lot worse than the other two approaches.

5 Discussion

Our experiments compare federated multi-task LoRA tuning under two aggregation designs: (i) an MTL-LoRA extension with stacked parameters and matrix-valued shared-knowledge weights (Approach I, RoBERTa) and (ii) an exact client-wise stacking construction with scalar mixture weights (Approach II, TinyLlama). Overall, aggregation behavior depends strongly on (a) the degree of client heterogeneity (IID vs. non-IID) and (b) the backbone architecture (encoder vs. decoder-only). Within their respective comparison groups, approach II is the most consistently beneficial method: on TinyLlama it achieves the strongest average performance, yields the best IID accuracy, and even slightly surpasses the centralized baseline, suggesting that federated optimization can act as a mild regularizer in this regime. Under non-IID splits, it remains clearly ahead of FedIT, indicating improved robustness to heterogeneity for LLM-like (decoder-only) backbones. In contrast, the RoBERTa results highlight pronounced model dependence: FedIT is surprisingly strong on average and outperforms approach I in IID setting and MRPC in non-IID settings. But under non-IID splits approach I performs clearly better on SST2 than FedIT. This task dependence is consistent with a task-wise analysis: under IID splits, most methods cluster tightly on SST2 and differences are largely driven by MRPC, whereas under non-IID splits the separation becomes much stronger and is often dominated

by SST2, where stacking-based methods preserve accuracy substantially better than FedIT (while MRPC can vary differently), implying that aggregate averages can mask qualitatively distinct failure modes. Conceptually, Approach II is also easier to justify scientifically because it directly implements an exact, no-cross-term aggregation target and keeps the task-mixture mechanism closer to the original MTL-LoRA formulation (scalar weights with client-wise normalization), whereas Approach I increases flexibility via matrix-valued weights that may amplify optimization instability and sensitivity to non-IID splits when multiple coupled tensors (A, B^i, Λ_t, and w_t^i) must be aligned across clients. A practical limitation of approach II is rank growth across rounds, increasing memory and communication costs. The linear-freeze variant (FreezeR) mitigates this by treating previously aggregated blocks as a frozen prefix and appending only new rank blocks per round, and empirically remains close to the best-performing configuration while improving scalability, though additional non-IID FreezeR results are needed to fully characterize this trade-off. Furthermore, it is useful to state explicitly that aggregation strategy conclusions appear *model-dependent*: stacking-based methods are most beneficial in the decoder-only LLM-like setting (TinyLlama), whereas RoBERTa shows stronger sensitivity under non-IID and benefits more from the stronger inductive bias of simpler averaging (FedIT). This directly connects to the FLoRA limitation that evidence is largely LLaMA-family focused, and motivates broader cross-architecture evaluation as future work. Finally, we follow prior work by reporting validation-only metrics, which preserves comparability but can bias hyperparameter selection. Future work should include a more diverse hyperparameter Study and test-set reporting, expand to more tasks and clients, and systematically evaluate cost–performance trade-offs for exact stacking vs. freezed R under both IID and non-IID splits.

6 Conclusion

In this work, we presented **MTL-FLoRA**, a federated extension of MTL-LoRA [2] that enables *multi-task* parameter-efficient fine-tuning while supporting *heterogeneous client ranks*. Building on the aggregation-noise insight of FLoRA [3], we proposed two server-side aggregation designs for the MTL-LoRA parameter set (A, Λ_t, B^i, w_t): (i) a stacking strategy that jointly aggregates all tensors while allowing *matrix-valued* shared-knowledge weighting (Approach I), and (ii) an *exact client-wise stacking* construction with scalar mixture weights and block-wise normalization (Approach II). For both designs, the key principle is to stack clients into disjoint rank blocks (with block-diagonal Λ_t and aligned A/B slices), thereby avoiding spurious cross terms and matching the intended global update under weighted client contributions.

Empirically, we evaluated federated multi-task tuning on GLUE tasks (MRPC and SST2) under IID and non-IID splits, comparing against centralized training and FedIT [1]. The results highlight a strong dependence on data heterogeneity and backbone architecture. On the decoder-only TinyLlama backbone, ap-

proach II consistently achieved the best average performance, was most robust under non-IID splits, and in our best configuration even slightly outperformed the centralized baseline—suggesting that, in this regime, federated optimization can act as a mild regularizer. On the encoder-only RoBERTa backbone, the picture is more mixed: FedIT remains a strong baseline (especially under non-IID), while stacking-based variants can be competitive and sometimes advantageous on SST2 but degrade more noticeably on MRPC under heterogeneity. Across models, we observed that the aggregation scaling factor p and the effective rank are critical hyperparameters: optimal values are configuration-dependent and must be tuned carefully to avoid performance collapse.

A practical limitation of approach II is *rank growth* across rounds, which increases communication and memory costs. Our linear-freeze ablation mitigates this by freezing previously aggregated blocks and appending only small new rank slices per round. It remains competitive while substantially improving scalability. Future work should (i) extend evaluation to more tasks and more clients, (ii) incorporate test-set reporting and broader hyperparameter sweeps to better characterize generalization and robustness across architectures (iii) evaluate the performance of both approaches on both models.

Acknowledgments. We would like to thank our supervisors for their support and commitment to this paper. Special thanks to Apoorva Upadhyaya, who kindly mentored our project.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

Retrospective We found it especially interesting to come up with ideas for applying different aggregation strategies to MTL-LoRA. Adapting FLoRA stacking to the MTL-LoRA parameters turned out more complicated than expected. Throughout the project, we came up with many ideas of how the stacking could be implemented so that the weighted sum of the shared knowledge with the Bw weights is combined meaningfully. We are looking forward to further work on this project, as we now have many ideas on how to further test and finegrain the approaches.

Note on the Use of AI Assistance We hereby declare that, in preparing this paper, we used AI-assisted tools, in particular GitHub Copilot (in Visual Studio Code) and OpenAI's ChatGPT. These tools served as sources of inspiration and provided peer-like, interactive support for ideation, discussing solution approaches, and feedback on code and text. We also used them for support with formatting, phrasing options, and proofreading. Autonomous generation or verbatim inclusion of text passages, code, analyses, or structure was not the aim of their use. Any incorporated material was reviewed, adapted, and integrated by me into the overall context of the paper.

References

1. Zhang, J., Vahidian, S., Kuo, M., Li, C., Zhang, R., Yu, T., Wang, G., Chen, Y.: Towards Building the Federated GPT: Federated Instruction Tuning. In: ICASSP 2024 – 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 6915–6919. IEEE, Seoul, Republic of Korea (2024). https://doi.org/10.1109/ICASSP48485.2024.10447454. https://arxiv.org/abs/2305.05644
2. Yang, Y., Muhtar, D., Shen, Y., Zhan, Y., Liu, J., Wang, Y., Sun, H., Deng, D., Sun, F., Zhang, Q., Chen, W., Tong, Y.: MTL-LoRA: Low-Rank Adaptation for Multi-Task Learning. arXiv preprint arXiv:2410.09437 (2025). https://arxiv.org/abs/2410.09437, last accessed 2026/02/10
3. Wang, Z., Shen, Z., He, Y., Sun, G., Wang, H., Lyu, L., Li, A.: FLoRA: Federated Fine-Tuning Large Language Models with Heterogeneous Low-Rank Adaptations. arXiv preprint arXiv:2409.05976 (2024). https://arxiv.org/abs/2409.05976, last accessed 2026/02/11
4. Author: Edward J. Hu and Yelong Shen and Phillip Wallis and Zeyuan Allen-Zhu and Yuanzhi Li and Shean Wang and Lu Wang and Weizhu Chen: LoRA: Low-Rank Adaptation of Large Language Models (2021), https://arxiv.org/abs/2106.09685, last acessed 2026/02/14
5. Author: H. Brendan McMahan and Eider Moore and Daniel Ramage and Seth Hampson and Blaise Agüera y Arcas: Communication-Efficient Learning of Deep Networks from Decentralized Data (2023), https://arxiv.org/abs/1602.05629
6. Author: H. Brendan McMahan and Eider Moore and Daniel Ramage and Seth Hampson and Blaise Agüera y Arcas: Communication-Efficient Learning of Deep Networks from Decentralized Data (2023), https://arxiv.org/abs/1602.05629

7 Appendix

7.1 Best Parameter Configurations for the approaches tested on Tinyllama

- Approach II: 3 Epochs, 3 FL-Rounds, 3 B-Matrices, Factor P = 1.0, $\alpha = 10$, 8 Ranks
- Approach II with freezed R: 3 Epochs, 3 FL-Rounds, 3 B-Matrices, Factor P = 1.5, $\alpha = 10$, 8 Ranks
- averaged-w: 3 Epochs, 3 FL-Rounds, 3 B-Matrices, Factor P = 1.0, $\alpha = 10$, 8 Ranks
- FedIT: 3 Epochs, 3 FL-Rounds, 3 B-Matrices, Factor P = 1.0, $\alpha = 10$, 8 Ranks
- Centralized:3 Epochs, 3 FL-Rounds, 3 B-Matrices, 8 Ranks

7.2 Best Parameter Configurations for the approaches tested on RoBERTa

- stacked-w: 3 Epochs, 3 FL-Rounds, 4 Ranks, 2 B-Matrices, Factor P = 1.0
- averaged-w: 3 Epochs, 3 B-Matrices, 3 FL-Rounds, 8 Ranks, Factor P = 1.25
- FedIT: 3 Epochs, 3 FL-Rounds, 3 B-Matrices, 4 Ranks
- Centralized: 4 Epochs, 3 B-Matrices, 8 Ranks

7.3 Training parameters (Same across all models)

- gradient accumulation steps 2
- learning rate $2e^{-4}$
- warmup ratio 0.06